INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

Trabajo de consulta

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Enfoque del Grupo 01: monitoreo operativo, potencia en el tiempo y detección de fallas

Presentado por
Cristian David López Cardona

Inteligencia de Negocios · Grupo 01 · Semestre 2026-II
Docente: Ramiro Grisales Montoya
Repositorio: https://github.com/Cat0428/solarbi-lopez-cardona

Medellín, octubre de 2026

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana y cuáles son sus componentes?

Grafana es una herramienta de código abierto para ver y vigilar datos que cambian en el tiempo. No guarda copia de nada: en cada refresco le pregunta a la fuente (Grafana Labs, 2026a). El **data source** es la conexión a la base; en SolarBI es PostgreSQL. Cada **panel** es un gráfico con su propia consulta, y varios paneles juntos forman un **dashboard**. Las **variables** son listas desplegables que cambian la consulta sin editarla, como $dispositivo. El **alerting** revisa reglas cada cierto tiempo y avisa cuando algo falla. **Grafana OSS** es gratuita y la instalé con Docker; **Grafana Cloud** es la versión alojada por Grafana Labs, con un plan gratuito limitado y planes pagos (Grafana Labs, 2026b).

D2. Tabla comparativa Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Analizar el negocio: modelar, cruzar datos y explicar resultados por periodos.	Monitorear la operación: ver el estado del sistema en este momento.
Usuario típico	Analista de BI, directivo, product owner.	Operador de planta, mantenimiento, equipos de TI.
Conexión a los datos	Importa una copia comprimida o consulta en vivo (DirectQuery); más de 100 conectores con Power Query.	Consulta la fuente en cada refresco; no guarda datos propios.
Modelo semántico y lenguaje	Modelo tabular con relaciones; medidas en DAX y transformaciones en M.	No tiene modelo semántico: cada panel usa el lenguaje de la fuente (SQL, PromQL, Flux).
Actualización y tiempo real	Actualización programada: 8 veces al día con Pro y 48 con Premium o PPU.	Casi en tiempo real: el dashboard se refresca cada pocos segundos.
Alertas	Alertas de datos solo en mosaicos de dashboards del servicio; en informes, Fabric Activator.	Alerting integrado: reglas sobre consultas, contact points y políticas de notificación.
Licenciamiento y costo	Desktop gratis; Pro a US$14 y PPU a US$24 por usuario al mes.	OSS gratis e instalada por uno; Cloud con plan gratuito y planes pagos.
Control de acceso	Roles del área de trabajo y seguridad a nivel de fila (RLS) con DAX.	Organizaciones, equipos y permisos por carpeta o dashboard; sin RLS propio.
Versionado	El .pbix es binario; el formato .pbip lo guarda como texto apto para Git.	El dashboard es un JSON que se exporta, se versiona y se carga por provisioning.
Instalación	Desktop solo en Windows; se publica en la nube de Microsoft.	Windows, Linux o Docker, y se usa desde el navegador.

Fuentes: Microsoft (2026a, 2026b, 2026c); EPC Group (2026); Grafana Labs (2026a, 2026b, 2026d).

Después de usar las dos con los mismos datos, mi conclusión es que no compiten. Grafana contesta «¿qué está pasando en la planta ahora mismo?» y Power BI contesta «¿cómo nos fue este mes y por qué?». Lo importante es que ambas leen la misma bodega en PostgreSQL.

D3. Dashboards operativo, analítico y estratégico

Few (2006) separa los tableros según quién los usa y para qué. El **operativo** es para quien está en la planta: muestra lo que pasa en este momento para reaccionar rápido. El **analítico** sirve para explorar, comparar periodos y buscar causas; lo usa el analista. El **estratégico** muestra pocos indicadores frente a metas y lo revisa la dirección cada mes. Con esa idea clasifiqué los KPI de SolarBI:

KPI	Tipo	Herramienta	Por qué
Energía (kWh)	Analítico / estratégico	Power BI (Grafana para el día en curso)	Se suma por día y por mes y se compara con metas; en Grafana basta un stat con la energía de hoy.
Yield y Performance Ratio	Analítico	Power BI	Hay que cruzar energía, capacidad e irradiación por periodo, y eso se resuelve con medidas DAX.
Ahorro (COP y CO_2)	Estratégico	Power BI	Es el argumento económico para la dirección y se reporta cada mes.
Disponibilidad y alarmas	Operativo	Grafana	Una falla hay que verla en minutos y avisarla; para eso están las alertas de Grafana.
% de datos válidos	Operativo y de gobierno	Grafana y Power BI	Grafana avisa si hoy la fuente falla; Power BI muestra cómo evoluciona la calidad.

D4. Grafana con PostgreSQL: consultas de series de tiempo

Para que Grafana dibuje una serie de tiempo, la consulta tiene que devolver una columna llamada time, al menos una columna numérica y los resultados ordenados por tiempo; si hay columnas de texto, Grafana las usa para separar series (Grafana Labs, 2026c). Las macros ahorran trabajo. **$__timeFilter(ts)** se cambia sola por un BETWEEN con el rango que el usuario eligió en el dashboard, así la base solo lee lo que se va a mostrar. **$__timeGroup(ts, '1h')** arma la expresión para agrupar por intervalos dentro de PostgreSQL; $__timeGroupAlias hace lo mismo y además nombra la columna time. Esta es la consulta que usé para la potencia promedio por hora:

```
SELECT $__timeGroupAlias(ts, '1h'),
      avg(p_ac_kw) AS "Potencia promedio (kW)"
FROM silver.lectura_5min
WHERE $__timeFilter(ts)
  AND dispositivo_id IN (${dispositivo})
GROUP BY 1
ORDER BY 1;
```

D5. Alertas: Grafana Alerting frente a Power BI

En **Grafana** una alerta tiene dos partes. La *alert rule* corre una consulta cada cierto tiempo y pasa por los estados Normal, Pending y Firing según la condición; el **contact point** define a dónde llega el aviso: correo, Telegram o Teams (Grafana Labs, 2026d). En **Power BI**, las alertas de datos solo existen en el servicio, sobre tarjetas o medidores de un dashboard, y se revisan cuando el modelo se actualiza, hasta ocho veces al día con Pro (Microsoft, 2026b). Para una planta es muy lento. Mi propuesta es la regla «Potencia cero en horario solar»: si el inversor marca 0 kW entre las 9:00 y las 15:00 durante cinco minutos, avisar por Telegram al grupo de operadores.

Bloque 2 · Gobernanza de datos (data governance)

G1. Data governance, data management y data quality

DAMA International (2017) define la **gobernanza de datos** como el ejercicio de autoridad y control sobre los datos: quién decide, con qué reglas y quién responde. La **gestión de datos** es más amplia e incluye el trabajo práctico: modelar, integrar, almacenar, proteger y analizar. En la rueda del DMBOK la gobernanza va en el centro y orienta lo demás. La **calidad de datos** es una de esas áreas y mide si el dato sirve para lo que se necesita. En SolarBI: gobernar es decidir que la potencia no puede ser negativa y quién responde por esa regla; gestionar es el ETL que la aplica, y medir calidad es saber que el 96,2 % de las lecturas esperadas fue válido.

G2. Roles data owner, data steward y data custodian

El **data owner** es el dueño del dato desde el negocio: decide quién lo ve, para qué y qué calidad se exige. El **data steward** lo cuida en el día a día: define reglas y términos, revisa la calidad y resuelve inconsistencias. El **data custodian** se encarga de la parte técnica: servidores, permisos, respaldos y cargas (DAMA International, 2017). Aunque hice este trabajo solo, en el proyecto los roles se reparten así:

Rol de gobierno	Rol en SolarBI	Qué hace en el proyecto
Data owner	Product owner	Aprueba el contrato de datos, los KPI y quién puede ver cada tablero.
Data steward	BI analyst y data modeler	Define las reglas (potencia entre 0 y 5,5 kW, una lectura cada 5 minutos), el diccionario de datos y las medidas DAX.
Data custodian	Data engineer	Opera PostgreSQL y el ETL, crea el usuario de solo lectura y cuida que las claves queden en el .env.

G3. El contrato de datos como instrumento de gobernanza

El contrato de datos es un acuerdo escrito entre quien produce el dato (el datalogger) y quienes lo usan (el ETL y los tableros). Debería decir qué campos llegan y de qué tipo, en qué **unidades** (kW, W/m², °C y hora de Colombia), cada cuánto (**cada 5 minutos**, 288 lecturas al día), quién es el **responsable** y qué **calidad mínima** se espera, como un 95 % de lecturas válidas. Si la fuente cambia p_ac_kw por p_ac_w sin avisar, lo menos malo es que el ETL falle. Lo peor es que cargue vatios como si fueran kilovatios y nadie lo note. Por eso mi run_etl.py exige las columnas del contrato y, si falta alguna, se detiene.

G4. Seguridad a nivel de fila (RLS) en Power BI y su equivalente en Grafana

En Power BI, la **RLS** se configura en Desktop creando **roles** con un filtro DAX; el filtro viaja por las relaciones hasta la tabla de hechos. En el servicio se asignan personas a cada rol, y solo afecta a los visores (Microsoft, 2026d). En mi modelo creé el rol *Responsable Robledo* con [sitio] = "Sede Robledo"; una versión dinámica usaría USERPRINCIPALNAME() para filtrar por el correo de quien entra. **Grafana OSS no tiene RLS.** Allí el acceso se controla con **organizaciones**, **equipos** y **permisos por carpeta o dashboard** (Grafana Labs, 2026e). Para separar sitios usaría una carpeta por sitio. Una variable del dashboard no protege nada, porque cualquiera puede editar la consulta.

G5. Ley 1581 de 2012 y el manejo responsable de la información

La Ley 1581 de 2012 protege los **datos personales**, es decir, la información que se puede asociar a una persona (Congreso de Colombia, 2012). Lo que mide SolarBI (potencia, irradiancia, temperatura) describe un equipo y no a alguien, así que **no es dato personal**. El panorama cambia si el tablero muestra qué operador atendió cada alarma: su nombre sí es dato personal y además serviría para juzgar su

desempeño. Entonces la Institución necesitaría su autorización, decirle para qué se usa el dato y limitar quién lo ve. Mi postura: un código de turno en vez del nombre, ese detalle solo para supervisores con RLS y nunca subir credenciales ni datos personales a GitHub.

Bloque 3 · Automatización de procesos ETL

E1. ETL frente a ELT y las capas Bronze, Silver y Gold

En **ETL** los datos se transforman antes de llegar al destino; en **ELT** se cargan tal como vienen y se transforman ya adentro, casi siempre con SQL. En la arquitectura medallón (Databricks, s. f.), **Bronze** guarda el dato crudo, **Silver** lo deja limpio con reglas de calidad y **Gold** lo resume para el negocio. Mi flujo mezcla los dos enfoques: de Bronze a Silver es ETL con pandas, y de Silver a Gold es ELT con SQL dentro de PostgreSQL. **Power Query sí es una herramienta ETL**, porque extrae, transforma con M y carga al modelo. Pero vive dentro de un informe y no orquesta ni escribe en la bodega. Por eso dejé las reglas en run_etl.py.

E2. Carga completa, carga incremental e idempotencia

La **carga completa** borra todo y lo vuelve a cargar en cada corrida: es sencilla, pero se vuelve pesada cuando crecen los datos. La **carga incremental** solo procesa lo nuevo desde la última vez, por ejemplo desde el último ts cargado; es más eficiente, aunque hay que cuidar los datos que llegan tarde. Una carga es **idempotente** cuando correrla una o diez veces deja exactamente lo mismo, y así se puede reintentar sin miedo. En fact_energia_dia lo conseguí con la clave primaria (fecha_key, dispositivo_key), que no deja repetir día y dispositivo, y un **UPSERT** que actualiza la fila si ya existe (PostgreSQL Global Development Group, 2026). Corrí el ETL cuatro veces y los conteos no cambiaron.

```
INSERT INTO dwh.fact_energia_dia (fecha_key, dispositivo_key, energia_kwh, ...)
SELECT ... FROM silver.lectura_5min GROUP BY ...
ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE
SET energia_kwh = EXCLUDED.energia_kwh, ...;
```

E3. Tres formas de automatizar un flujo

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	El programador del sistema operativo: lanza un comando a una hora fija.	Una plataforma de orquestación donde los flujos se escriben como DAG en Python.	Una función del servicio de Power BI que recarga el modelo semántico.
Qué automatiza	Cualquier script, por ejemplo python etl/run_etl.py.	Tareas encadenadas con dependencias, reintentos y bitácora (simulador → ETL → pruebas → refresh).	Solo la recarga de datos del modelo; con fuentes locales necesita gateway.
A favor	Es simple y no hay que instalar nada.	Se ve qué falló, reintenta solo y guarda historial.	Viene incluida en el servicio.
En contra	No maneja dependencias, reintentos ni monitoreo.	Hay que aprenderla y montarle infraestructura.	8 actualizaciones al día con Pro y no ejecuta el ETL.
Cuándo conviene	Un solo proceso diario, como el de SolarBI hoy.	Varios pasos encadenados o varias plantas.	Siempre que el informe esté publicado, después del ETL.

Fuentes: Apache Software Foundation (2026); Microsoft (2026c).

Para SolarBI me quedo con cron, o el Programador de tareas de Windows, a medianoche para el ETL, y la actualización programada de Power BI una hora después, con gateway porque la base es local. Si el flujo crece, Airflow sería el paso siguiente.

E4. Query folding en Power Query

El **query folding** ocurre cuando Power Query logra traducir los pasos que uno aplica, como filtrar, quitar columnas o agrupar, a una sola consulta SQL que ejecuta PostgreSQL (Microsoft, 2026e). Así viajan menos datos, la base usa sus índices y Power BI gasta menos memoria; además, la actualización incremental lo necesita. El plegado se rompe con pasos que la base no entiende, como una función personalizada o Text.Proper. Para revisarlo, doy clic derecho sobre el paso y elijo **Ver consulta nativa**: si la opción está activa y muestra el SQL, el paso se pliega; si aparece en gris, no. También se puede ver desde PostgreSQL qué consulta llegó, en pg_stat_activity.

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

La primera capa es el **dispositivo**: el inversor y los sensores que miden. Luego está el **gateway**, un datalogger o una Raspberry Pi que lee los equipos cada cinco minutos, le pone la hora a cada lectura y la guarda si se cae el internet. La capa de **red y broker** lleva el mensaje por Wi-Fi o LTE hasta un broker MQTT, que lo reparte a quien lo necesite. En **almacenamiento**, la lectura llega cruda a Bronze, se limpia en Silver y se resume por día en Gold. Por último está la **aplicación**: Grafana lee Silver para el operador y Power BI lee Gold para la dirección. La Figura 1 muestra ese recorrido.

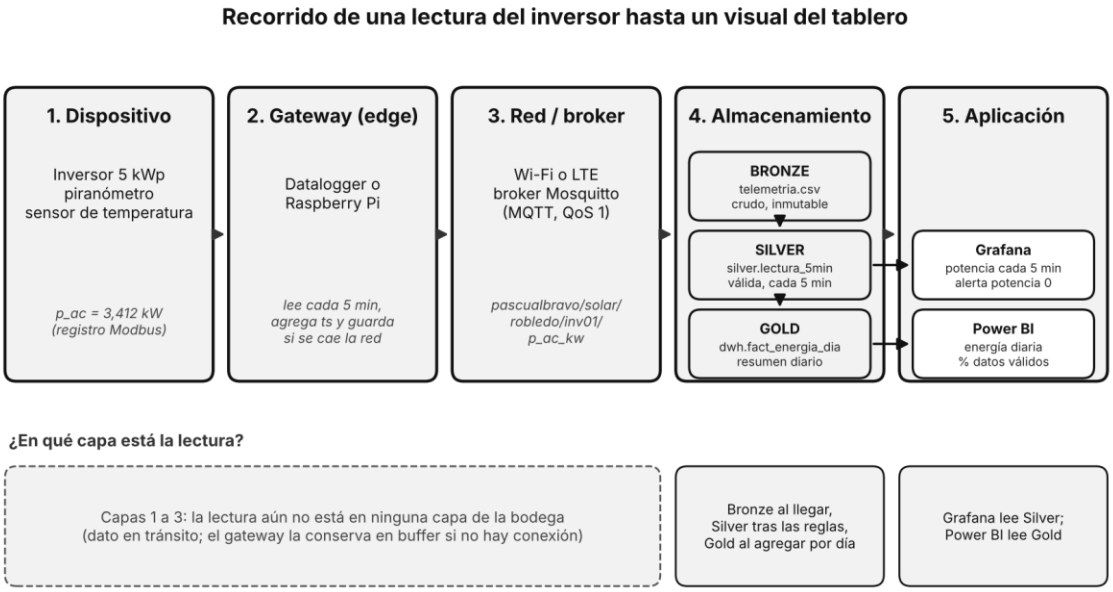


Figura 1. *Recorrido de una lectura del inversor hasta el tablero (elaboración propia).*

I2. MQTT: publish/subscribe, broker, topics y QoS

MQTT es un protocolo de mensajería muy liviano, estandarizado por OASIS (2019), pensado para equipos y redes limitados. Funciona con **publicación y suscripción**: el datalogger publica sin saber quién escucha, y cada consumidor se suscribe a lo que le interesa. En el medio está el **broker**, que recibe los mensajes y los reenvía. Cada mensaje va a un **topic**, una ruta con niveles separados por /. Hay tres niveles de entrega: **QoS 0** envía una vez y no confirma; **QoS 1** confirma, pero puede duplicar; **QoS 2** garantiza una sola

entrega, con más intercambio de mensajes. Para la telemetría elegiría QoS 1 y quitaría los duplicados en Silver, que es justo lo que hace la regla R1.

```
pascualbravo/solar/robledo/inv01/p_ac_kw          ← potencia AC (kW)
pascualbravo/solar/robledo/inv01/temp_modulo_c    ← temperatura (°C)
pascualbravo/solar/robledo/piranometro01/irradiancia_wm2
pascualbravo/solar/robledo/inv01/estado           ← retenido + Last Will "offline"
pascualbravo/solar/robledo/inv01/alarma           ← códigos de falla

Suscripciones: pascualbravo/solar/+/+/p_ac_kw   (todas las potencias)
               pascualbravo/solar/robledo/#     (todo un sitio)
```

I3. Pila abierta de monitoreo IoT

Componente	Papel en la pila
Mosquitto	El broker MQTT de Eclipse: recibe lo que publica el datalogger y lo reparte por topic.
Telegraf	Un agente de InfluxData: con mqtt_consumer se suscribe a los topics, convierte el JSON y lo escribe en la base.
InfluxDB o TimescaleDB	La base de series de tiempo: aguanta muchas escrituras por segundo y maneja retención y resúmenes. TimescaleDB es una extensión de PostgreSQL.
Grafana	Muestra los datos y lanza las alertas consultando esa base.

PostgreSQL puede entrar de dos maneras. Con **TimescaleDB**, la misma base guardaría la telemetría de cinco minutos y también el esquema estrella para Power BI, y Grafana agruparía con time_bucket. Con **InfluxDB**, PostgreSQL quedaría solo como bodega analítica alimentada por el ETL nocturno (InfluxData, 2026; Grafana Labs, 2026c).

I4. Procesamiento por lotes frente a flujo continuo

En el procesamiento **por lotes** los datos se acumulan y se procesan juntos a una hora fija, como mi ETL de medianoche: es simple, pero va con horas de retraso. En **streaming** cada dato se procesa al llegar, aunque montarlo es más complejo. Al pasar de cinco minutos a un dato diario, 288 filas se vuelven una, Power BI responde más rápido y KPI como energía o PR se comparan fácil. Se pierde la forma del día y la posibilidad de ver fallas: un corte de media hora al mediodía queda escondido en un total apenas más bajo, y no se puede desagregar después (Kimball y Ross, 2013). Por eso conservo Silver para Grafana y Gold para Power BI.

Bloque 5 · Control de versiones

V1. Git, GitHub y el versionado de tableros

Git lleva el historial de cambios en cada computador; **GitHub** es el sitio donde se publican los repositorios y se colabora (Chacon y Straub, 2014). Un **repositorio** es el proyecto con todo su historial; un **commit** es una foto de los cambios, con autor y mensaje; una **branch** es una línea de trabajo paralela, y un **pull request** pide unir una rama a main tras revisarla. El .pbix es binario: Git no puede mostrar qué cambió. El .pbip guarda el informe y el modelo como archivos de texto (TMDL y JSON) que sí se pueden comparar (Microsoft, 2026f); por eso subí los dos formatos. En Grafana, el dashboard es un JSON que se exporta y se versiona.

Parte B · Práctica guiada: los mismos datos en dos herramientas

En esta parte armé un flujo completo, aunque pequeño: datos crudos de un inversor simulado, limpieza con reglas de calidad, carga en PostgreSQL y dos tableros, uno en Power BI y otro en Grafana, que leen **la misma base**. El código completo está en el repositorio; aquí dejo las evidencias de cada paso.

Paso 1 · Bronze: datos crudos

Usé el simulador que entregó el docente (etl/simulador.py) con un solo cambio: le puse una semilla fija (2026) para que el archivo salga igual cada vez que se ejecute. Genera tres días, del 5 al 7 de octubre de 2026, de un inversor de 5 kWp con una lectura cada cinco minutos. Son 864 intervalos más 25 filas duplicadas, para un total de **889 filas**, con potencias negativas e irradiancias vacías metidas a propósito. El ETL solo lee este archivo; nunca lo modifica.

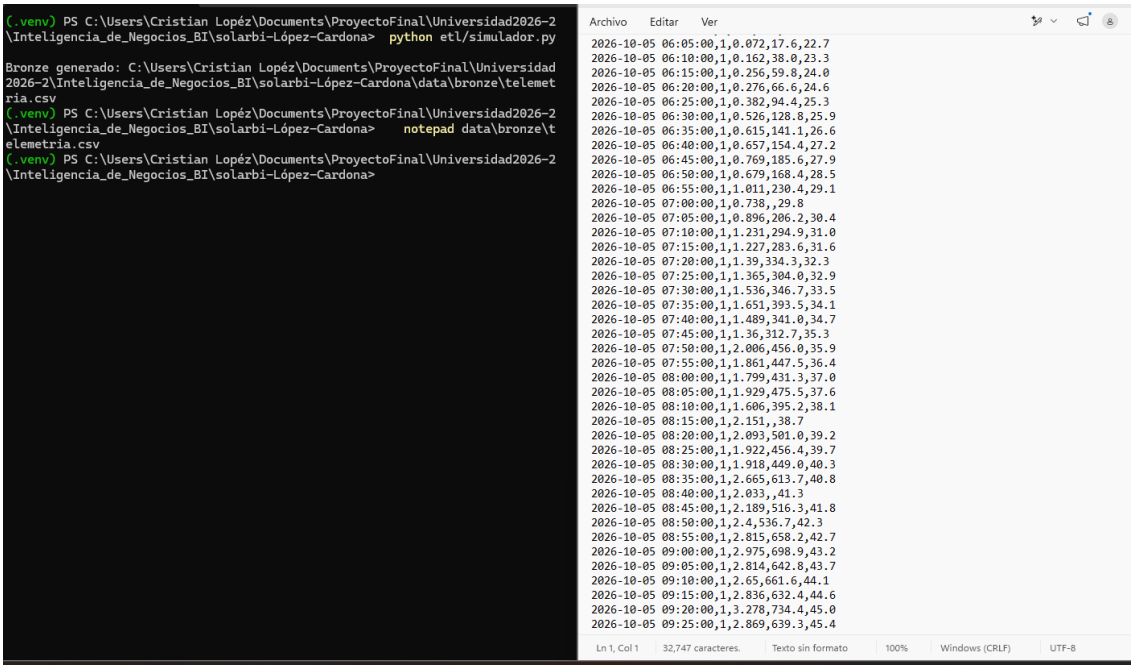


Figura 2. *Ejecución del simulador y primeras filas de data/bronze/telemetria.csv.*

Paso 2 · Silver: reglas de calidad

El script run_etl.py aplica tres reglas en orden con pandas. Las filas que no pasan quedan guardadas en data/silver/rechazos.csv con el motivo, y el resumen queda en data/silver/reporte_calidad.json.

Regla	Cuándo se rechaza una fila	Rechazadas
R1 · Duplicados	Se repite la pareja (ts, dispositivo_id); se conserva la primera.	25
R2 · Faltantes	Algún campo obligatorio viene vacío o no es un número.	23
R3 · Rango físico	Potencia fuera de 0 a 5,5 kW (110 % de 5 kWp), irradiancia fuera de 0 a 1500 W/m² o temperatura fuera de -10 a 90 °C.	10

Filas leídas	Rechazadas	Filas válidas	% válido sobre lo leído	% válido sobre lo esperado (864)
889	58	831	93,48 %	96,18 %

Muestro dos porcentajes porque miden cosas distintas. El primero dice qué tan limpio llegó el archivo; el segundo, qué tan completo quedó cada día frente a las 288 lecturas esperadas. Este último es el que uso en los tableros, porque un duplicado no es un dato perdido.



Figura 3. *Reporte de calidad que imprime run_etl.py.*

Paso 3 · Gold: carga en PostgreSQL

Las lecturas limpias van a silver.lectura_5min, con la energía de cada intervalo calculada como p_ac_kw × 5/60. Después, sql/03_carga_gold.sql las resume por día en dwh.fact_energia_dia, junto con las dimensiones dim_fecha y dim_dispositivo, y cada corrida queda anotada en la bitácora dwh.etl_ejecucion. Todo pasa en una sola transacción y con un solo comando: python etl/run_etl.py.

Fecha	Energía (kWh)	Yield (kWh/kWp)	Performance Ratio	Lecturas válidas	% datos válidos
05/10/2026	26,149	5,230	85,3 %	275 / 288	95,49 %
06/10/2026	26,535	5,307	85,3 %	279 / 288	96,88 %
07/10/2026	26,349	5,270	84,6 %	277 / 288	96,18 %
Total	79,033	15,807	85,1 %	831 / 864	96,18 %

Idempotencia. Ejecuté el ETL cuatro veces. En todas quedaron 831 filas en Silver y 3 en la tabla de hechos, porque cada escritura es un INSERT … ON CONFLICT … DO UPDATE sobre la clave primaria. La única tabla que crece es la bitácora, y sus cuatro registros tienen exactamente las mismas cifras.

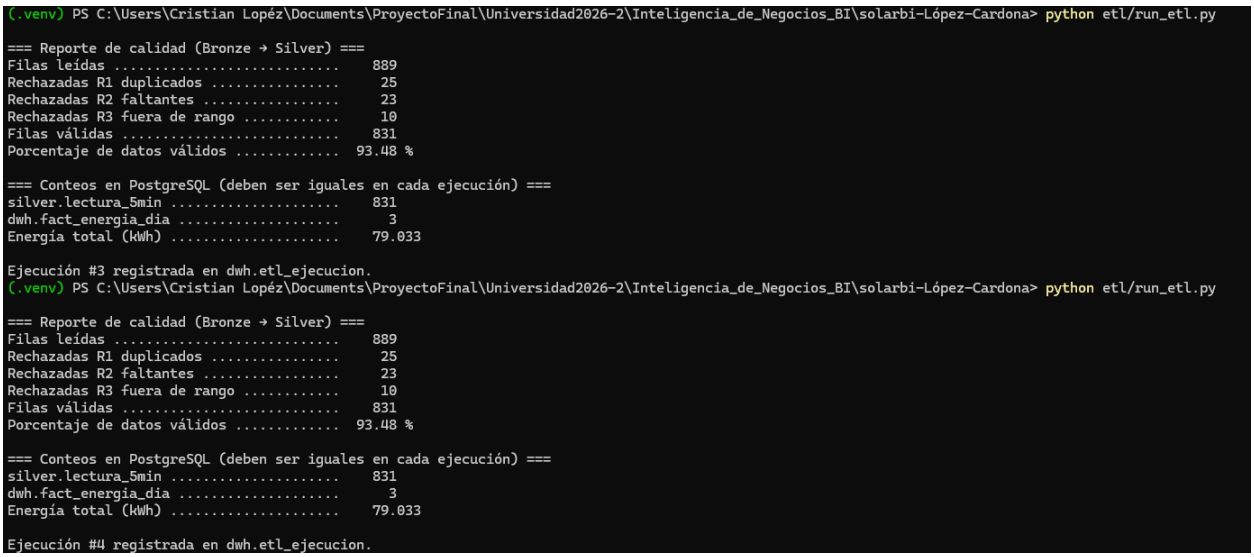


Figura 4. *Dos ejecuciones seguidas de run_etl.py con los mismos conteos.*

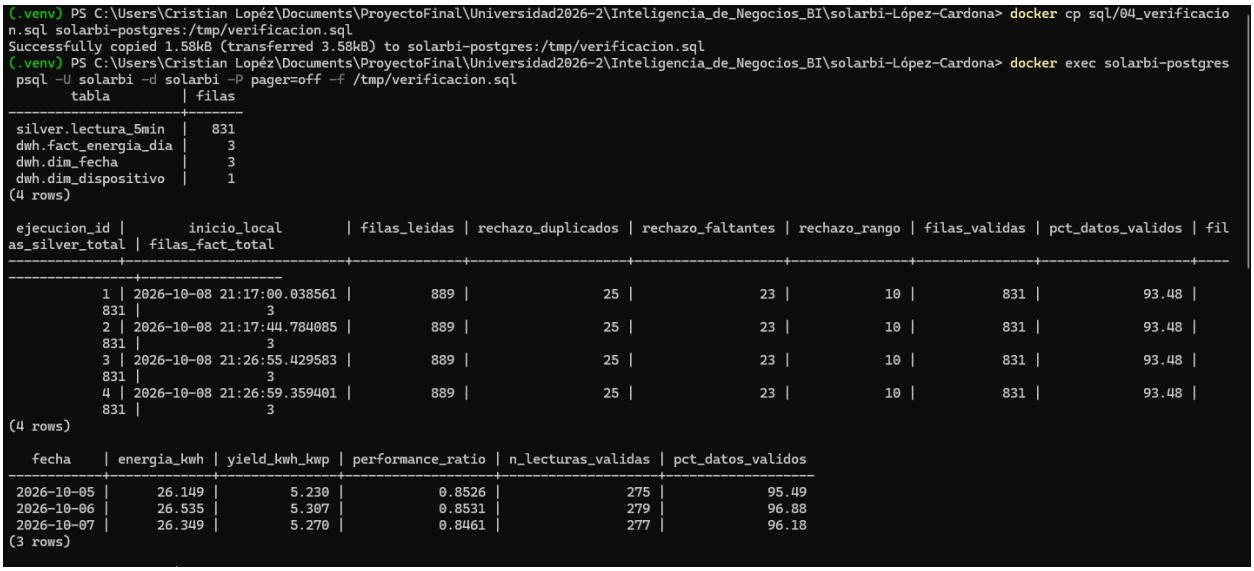


Figura 5. *Verificación en PostgreSQL: conteos por tabla, bitácora de ejecuciones y resumen diario.*

Programación diaria. No la dejé activa, pero así se lanzaría el ETL todos los días a medianoche, con cron o con el Programador de tareas de Windows:

```
0 0 * * *  cd /ruta/solarbi-lopez-cardona && .venv/bin/python etl/run_etl.py >> logs/etl.log 2>&1

schtasks /Create /TN "SolarBI ETL" /SC DAILY /ST 00:00 ^
        /TR "C:\ruta\solarbi\.venv\Scripts\python.exe C:\ruta\solarbi\etl\run_etl.py"
```

Paso 4 · Power BI

Conecté Power BI Desktop a PostgreSQL en modo importación con el usuario bi_lector, que solo puede leer. En mi equipo el servidor quedó como 127.0.0.1:55432, porque ya tenía otro PostgreSQL ocupando el puerto 5432, y tuve que desmarcar la conexión cifrada porque el contenedor no tiene SSL. Importé las tres tablas, ajusté las relaciones para que quedaran de muchos a uno y marqué dim_fecha como tabla de fechas. Las cifras de la página salen de medidas DAX:

```
Energía Total (kWh) = SUM ( fact_energia_dia[energia_kwh] )

% Datos Válidos = DIVIDE ( SUM ( fact_energia_dia[n_lecturas_validas] ),
                          SUM ( fact_energia_dia[n_lecturas_esperadas] ) )

Performance Ratio = DIVIDE ( SUM ( fact_energia_dia[yield_kwh_kwp] ),
                             SUM ( fact_energia_dia[irradiacion_kwh_m2] ) )
```

La tarjeta de energía muestra **79,03 kWh**, la de calidad **96,2 %**, y el gráfico de líneas enseña los tres días, entre 26,15 y 26,54 kWh. Dejé el eje vertical desde cero para no exagerar una diferencia que en realidad es pequeña. % Datos Válidos es un promedio ponderado: si promediara los porcentajes de cada día, el resultado cambiaría cuando los días tienen distinta cantidad de lecturas. En el modelo también quedó el rol de RLS *Responsable Robledo* de la pregunta G4, y guardé el informe como .pbix y como proyecto .pbip.

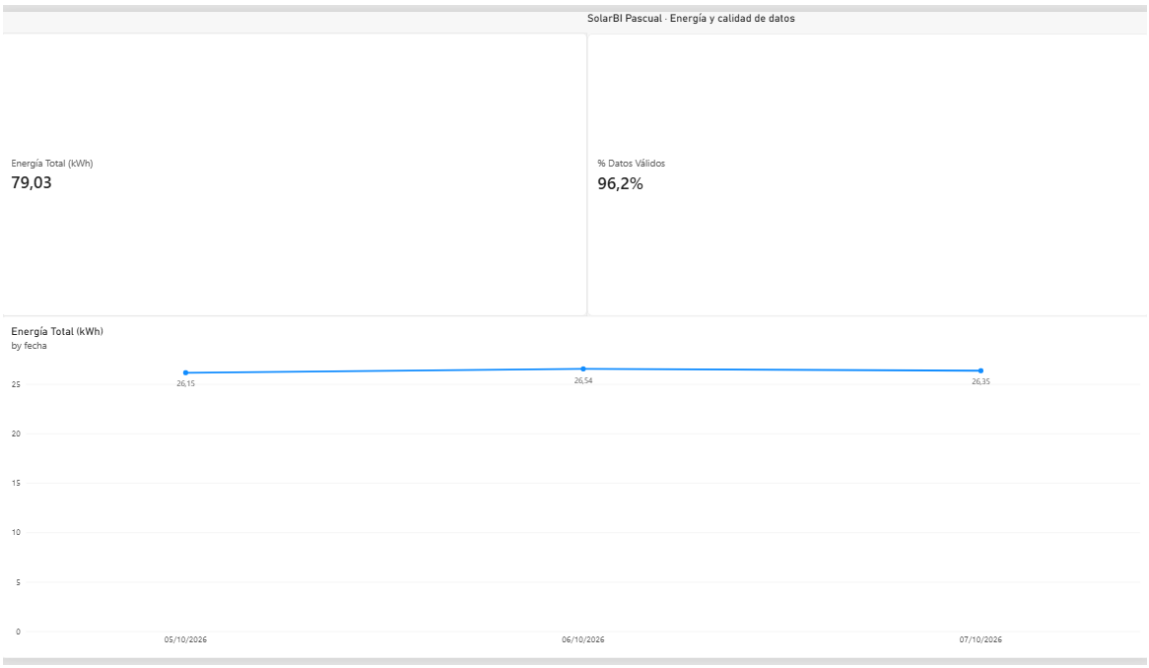


Figura 6. *Página de Power BI con la energía total, la energía diaria y el porcentaje de datos válidos.*

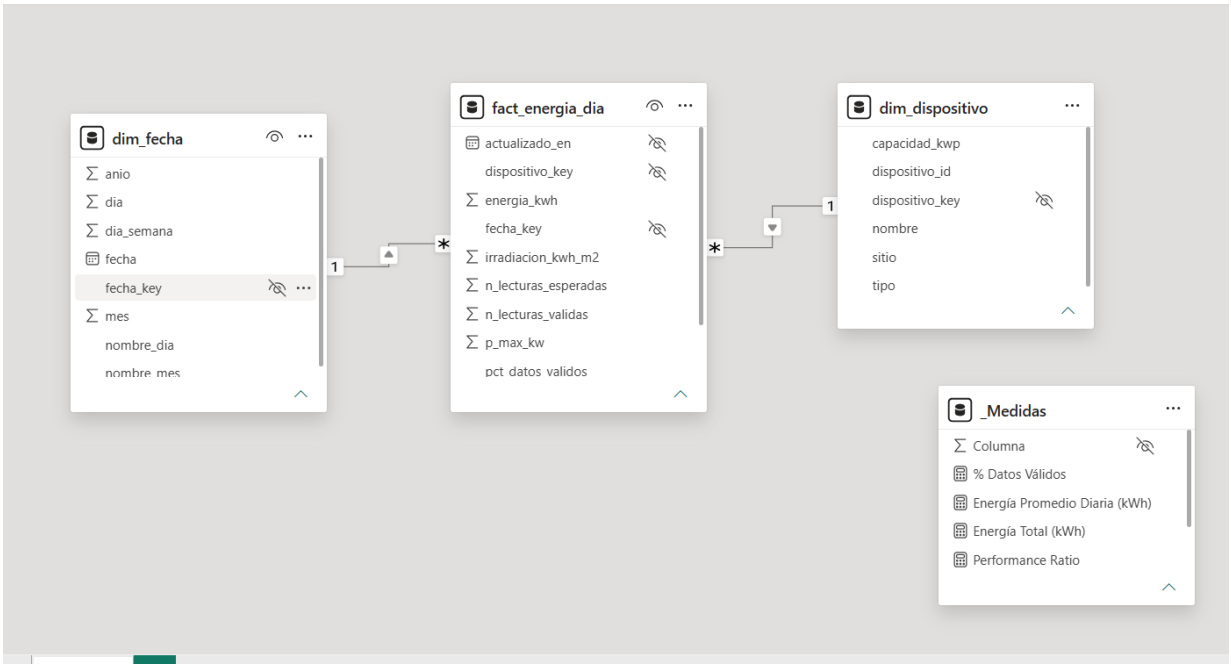


Figura 7. *Vista de modelo: esquema estrella y tabla de medidas.*

Paso 5 · Grafana

Levanté Grafana OSS 12 con Docker. Gracias al *provisioning*, al arrancar ya quedan creados el data source (también con bi_lector), el dashboard **SolarBI Pascual · Monitoreo operativo** y la regla de alerta. En mi equipo tuve que publicar Grafana en el puerto 3001, porque el 3000 estaba ocupado. El dashboard tiene una variable $dispositivo y siete paneles:

Panel	Tipo	Qué muestra
Potencia AC cada 5 minutos	Serie de tiempo	La potencia desde Silver con $__timeFilter, con un umbral punteado en 0,1 kW.
Potencia promedio por hora	Barras	La consulta de la pregunta D4, con $__timeGroupAlias(ts, '1h').
Energía del periodo (Silver)	Stat	La suma de energía del rango elegido; si se elige un día, es la energía de ese día.
Energía del día (Gold)	Stat	La misma cifra leída desde fact_energia_dia, para comprobar que coincide con Power BI.
Datos válidos	Stat	Lecturas válidas sobre lecturas esperadas.
Fallas 9:00–15:00	Stat	Cuántas lecturas tuvieron potencia 0 en horario de sol.
Energía diaria (Gold)	Barras	La energía de cada día desde la tabla de hechos.

```
SELECT ts AS "time", d.nombre AS dispositivo, l.p_ac_kw AS "Potencia AC"
FROM silver.lectura_5min l
JOIN dwh.dim_dispositivo d USING (dispositivo_id)
WHERE $__timeFilter(ts) AND l.dispositivo_id IN (${dispositivo})
ORDER BY 1
```

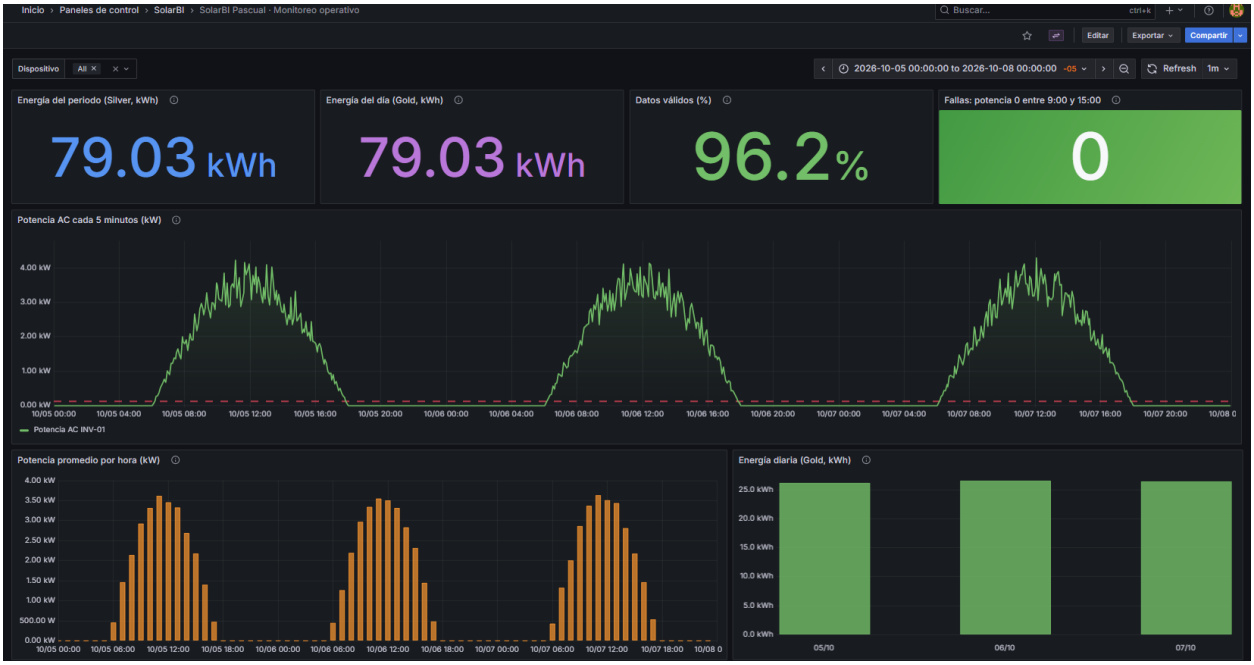


Figura 8. *Dashboard de Grafana entre el 5 y el 7 de octubre de 2026.*

Alerta. La regla *Potencia cero en horario solar (9:00–15:00)* revisa cada minuto si hay lecturas en 0 kW entre las 9:00 y las 15:00, hora de Colombia, y se dispara si la condición se mantiene cinco minutos. La primera vez quedó en error por un detalle en el archivo de la regla: las macros estaban escritas con $$ en lugar de $. Después de corregirlo inserté una lectura de prueba de 0 kW al mediodía (sql/06_prueba_alerta.sql), y la regla pasó de Normal a Pending y luego a Firing. Al terminar borré esa lectura. El dashboard quedó exportado como JSON en grafana/dashboard.json.

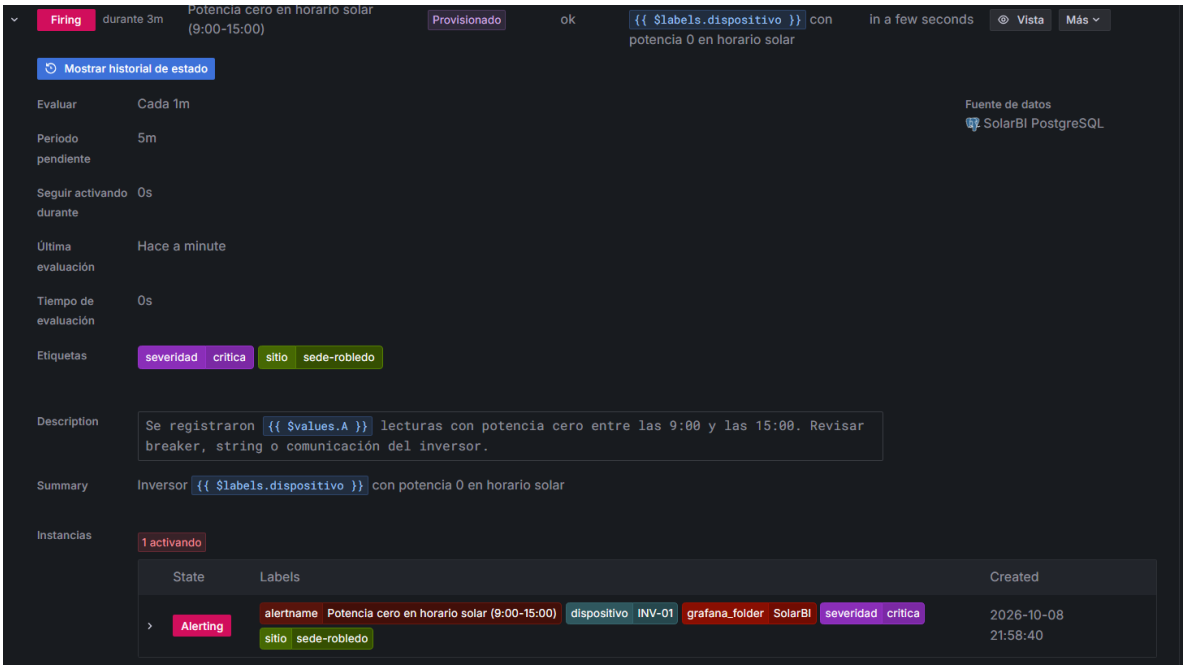


Figura 9. *Regla de alerta en estado Firing después de la prueba.*

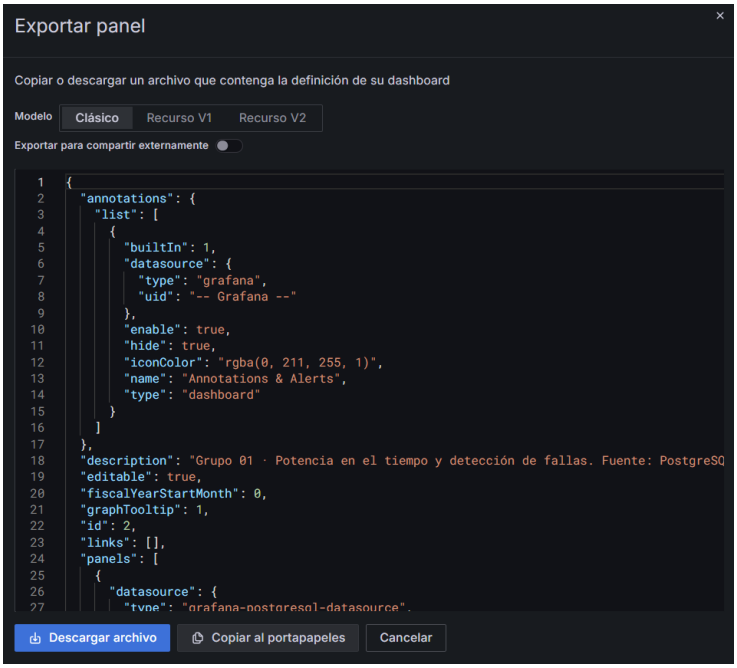


Figura 10. *Exportación del dashboard como JSON.*

Paso 6 · Reflexión

Con los mismos datos, Grafana fue mejor para seguir la potencia en el tiempo: con una consulta SQL tuve la curva de cada día, el umbral y una alerta que de verdad se disparó. Power BI fue mejor para resumir: con tres medidas DAX obtuve la energía, la calidad y el PR, y puedo filtrarlos sin escribir más SQL. Lo que más me costó no fue el análisis sino el entorno: los puertos 3000 y 5432 ya ocupados en mi equipo, la conexión cifrada y el «host desconocido» en Power BI, y la jerarquía automática de fechas que dejaba el gráfico vacío. Para el operador de la planta usaría Grafana, porque necesita enterarse de una falla en

minutos; para un directivo, Power BI, porque le interesa la tendencia y el resultado del mes. Al final, las dos muestran 26,54 kWh para el 6 de octubre: el mismo dato, visto de dos formas.

Parte C · Repositorio en GitHub

Enlace: https://github.com/Cat0428/solarbi-lopez-cardona

El repositorio es público y lo voy a seguir usando en los próximos entregables de SolarBI. En el README explico cómo reproducir la práctica de principio a fin y dejé una tabla con los problemas que encontré en Windows y cómo los resolví.

Carpeta o archivo	Contenido
README.md	Autor, curso y grupo, descripción, pasos para reproducir y problemas frecuentes.
docker-compose.yml, .env.example	PostgreSQL 16 y Grafana OSS listos para levantar; plantilla de variables sin claves reales.
data/bronze/, data/silver/	Telemetría cruda; lecturas válidas, rechazos y reporte de calidad.
etl/	simulador.py y run_etl.py, el punto de entrada único.
sql/	Tablas, usuario de solo lectura, carga Gold con UPSERT, verificación y consultas de la alerta.
powerbi/	El informe en .pbix y como proyecto .pbip, más la guía con las medidas DAX y la RLS.
grafana/	dashboard.json exportado y el provisioning del data source, el dashboard y la alerta.

Credenciales. En el repositorio no hay ninguna contraseña. El ETL, Docker y Grafana las leen de un archivo .env que está excluido en el .gitignore, y los tableros se conectan con bi_lector, que no puede modificar datos.

Trabajo individual. Desarrollé este trabajo solo, así que no aplican la invitación de un colaborador ni el reparto de commits entre dos personas. En su lugar muestro el repositorio publicado y el historial de commits.

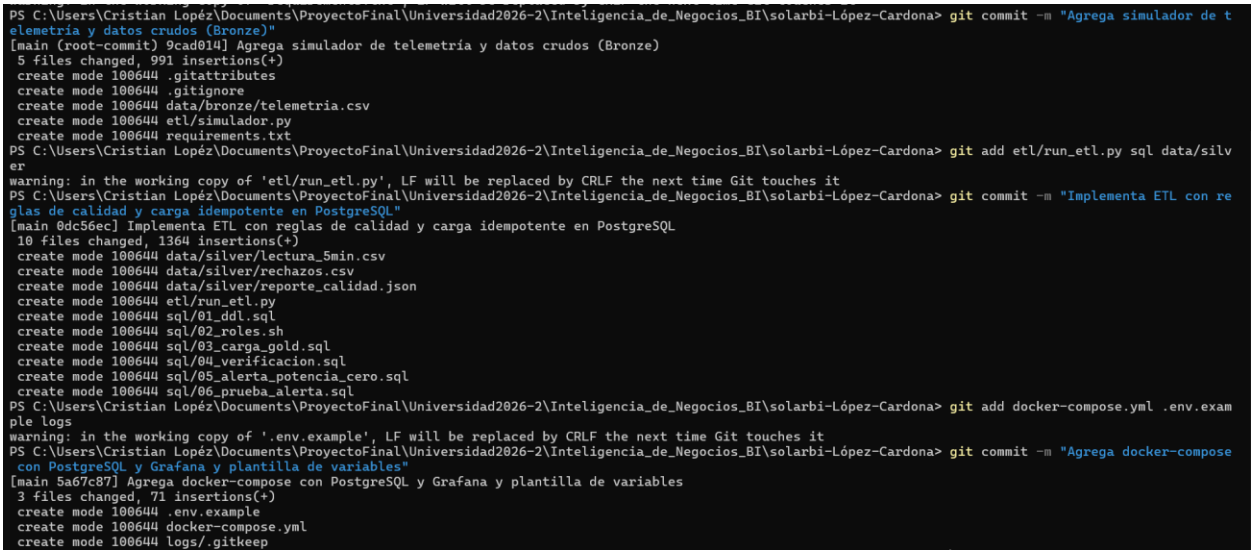


Figura 11. *Repositorio publicado en GitHub con las carpetas del proyecto.*

```
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git add grafana
warning: in the working copy of 'grafana/dashboard (2).json', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git commit -m "Agrega dashboard de G
rafana exportado y provisioning de la alerta"
[main b3f8a61] Agrega dashboard de Grafana exportado y provisioning de la alerta
 4 files changed, 754 insertions(+)
 create mode 100644 grafana/dashboard (2).json
 create mode 100644 grafana/provisioning/alerting/reglas.yaml
 create mode 100644 grafana/provisioning/dashboards/solarbi.yaml
 create mode 100644 grafana/provisioning/datasources/postgres.yaml
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git add powerbi
warning: in the working copy of 'powerbi/README.md', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git commit -m "Agrega informe de Pow
er BI en formato PBIP con medidas DAX y RLS"
[main f4e214a] Agrega informe de Power BI en formato PBIP con medidas DAX y RLS
 2 files changed, 86 insertions(+)
 create mode 100644 powerbi/README.md
 create mode 100644 powerbi/solarbi.pbix
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona> git commit -m "Completa README con p
asos de reproducción"
[main 8afabd2] Completa README con pasos de reproducción
 1 file changed, 130 insertions(+)
 create mode 100644 README.md
PS C:\Users\Cristian Lopéz\Documents\ProyectoFinal\Universidad2026-2\Inteligencia_de_Negocios_BI\solarbi-López-Cardona>
```

Figura 12. *Historial de commits del repositorio.*

Referencias

Apache Software Foundation. (2026). *Apache Airflow documentation*. https://airflow.apache.org/docs/apache-airflow/stable/index.html

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Congreso de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.º 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2.ª ed.). Technics Publications.

Databricks. (s. f.). *Medallion architecture*. Recuperado el 7 de octubre de 2026 de https://www.databricks.com/glossary/medallion-architecture

EPC Group. (2026). *Power BI pricing and licensing 2026*. https://www.epcgroup.net/insights/power-bi-pricing-licensing-2026

Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly Media.

Grafana Labs. (2026a). *Dashboards* [Documentación de Grafana]. https://grafana.com/docs/grafana/latest/dashboards/

Grafana Labs. (2026b). *What's included in Grafana Cloud Free*. https://grafana.com/products/cloud/free-tier/

Grafana Labs. (2026c). *PostgreSQL query editor* [Documentación de Grafana]. https://grafana.com/docs/grafana/latest/datasources/postgres/query-editor/

Grafana Labs. (2026d). *Grafana Alerting* [Documentación de Grafana]. https://grafana.com/docs/grafana/latest/alerting/

Grafana Labs. (2026e). *Roles and permissions* [Documentación de Grafana]. https://grafana.com/docs/grafana/latest/administration/roles-and-permissions/

InfluxData. (2026). *Telegraf documentation*. https://docs.influxdata.com/telegraf/v1/

Kimball, R., y Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.ª ed.). Wiley.

Microsoft. (2026a). *¿Qué es Power BI?* Microsoft Learn. https://learn.microsoft.com/es-es/power-bi/fundamentals/power-bi-overview

Microsoft. (2026b). *Set data alerts in the Power BI service*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/create-reports/service-set-data-alerts

Microsoft. (2026c). *Configure scheduled refresh*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh

Microsoft. (2026d). *Row-level security (RLS) with Power BI*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/enterprise/service-admin-rls

Microsoft. (2026e). *Query folding overview*. Microsoft Learn. https://learn.microsoft.com/en-us/power-query/query-folding-basics

Microsoft. (2026f). *Power BI Desktop projects (PBIP)*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview

OASIS. (2019). *MQTT version 5.0* (OASIS Standard). https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

PostgreSQL Global Development Group. (2026). *INSERT* [PostgreSQL 16 documentation]. https://www.postgresql.org/docs/16/sql-insert.html